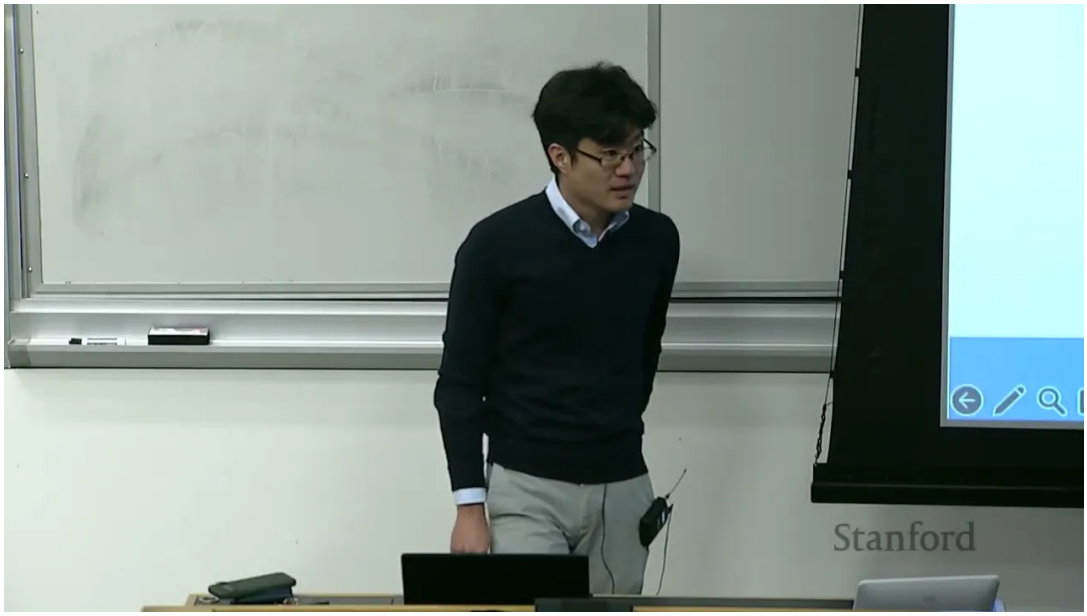
Stanford CS336 中文讲义

Language Modeling from Scratch
Lecture 5: GPUs

从 GPU 基础到 FlashAttention 的系统化理解

整理：Codex

日期：2026-03-24



视频标题： Stanford CS336 I Language Modeling from Scratch | Spring 2025 | Lecture 5: GPUs
频道： Stanford Online
发布日期： 2025-05-01
时长： 1:14:21
链接： https://www.youtube.com/watch?v=6OBtO9niT00

目录

1 导论：为什么学大模型必须懂 GPU **2**

1.1 从算力扩展谈起 2

1.2 本章小结 3

2 GPU 与 CPU：设计目标完全不同 **3**

2.1 为什么这对大模型如此重要 4

2.2 本章小结 5

3 GPU 的内存层次与执行模型 **5**

3.1 从物理距离理解内存速度 5

3.2 Blocks、Warps、Threads 5

3.3 一个实用公式：Roofline 直觉 6

3.4 本章小结 7

4 让 GPU 真正跑快的四类技巧 **7**

4.1 技巧一：算子融合与减少中间写回 7

4.2 技巧二：重计算（Recomputation） 8

4.3 技巧三：Memory Coalescing 9

4.4 技巧四：Tiling 9

4.5 波数量化（Wave Quantization） 10

4.6 本章小结 11

5 FlashAttention：把所有原则装进一个算法 **11**

5.1 FlashAttention 的硬件视角 11

5.2 为什么 softmax 是难点 11

5.3 FlashAttention 如何对应前文四大技巧 12

5.4 一个简化的 tiled attention 伪代码 12

5.5 本章小结 12

6 总结与延伸 **12**

6.1 拓展阅读 13

1 导论：为什么学大模型必须懂 GPU

这节课的核心目标有两个。第一，讲者希望把“CUDA 和 GPU 没那么神秘”这件事讲清楚，让学习者知道 GPU 为什么会在某些矩阵规模上特别快、在另一些规模上又突然变慢。第二，讲者希望把这些硬件与执行模型知识，最后落到一个真正重要的算法案例上，也就是 FlashAttention。换句话说，这节课不是单纯介绍硬件名词，而是试图解释：为什么这些名词会决定大模型训练与推理的实际性能。

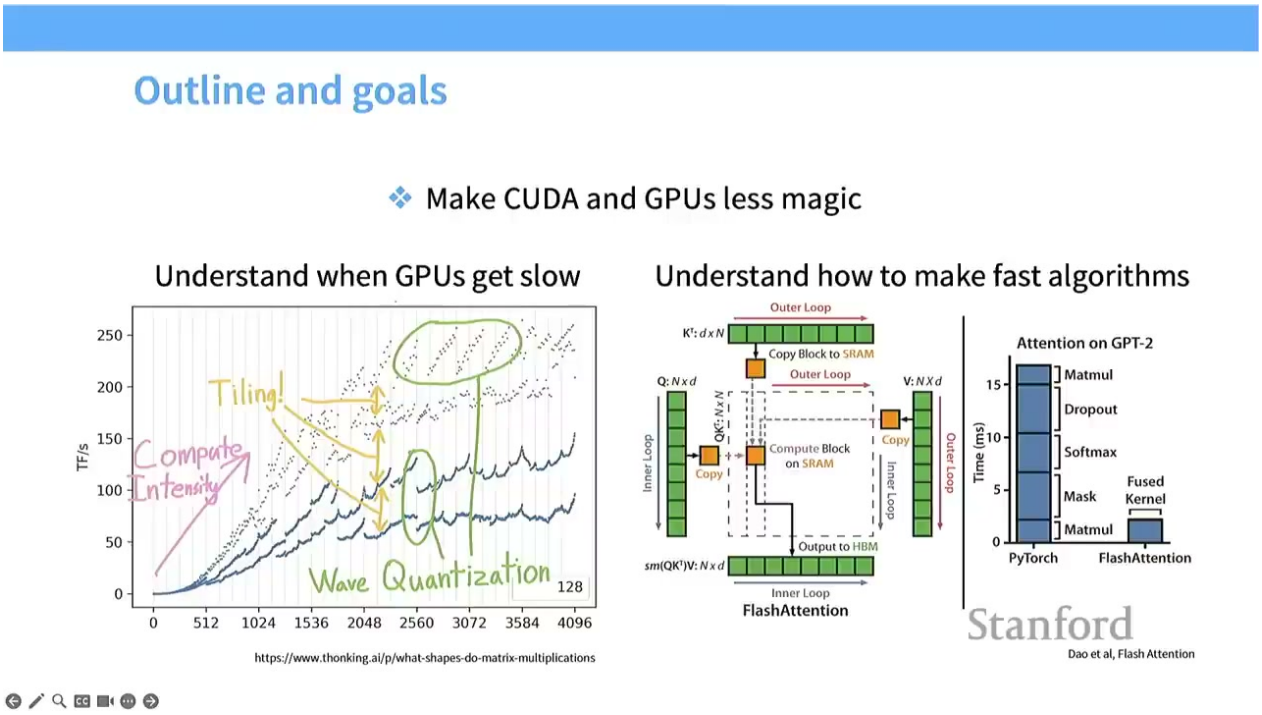


图 1: 课程开场：从 GPU 性能波动到 FlashAttention 的全讲目标

来源时间：00:00:44–00:01:40。

本讲的总问题

讲者真正关心的不是“GPU 有哪些部件”，而是两个更实用的问题：为什么 GPU 会慢，以及如何设计出像 FlashAttention 这样真正快的算法。

在大模型背景下，这个问题特别重要。课程开头先把背景摆得很明确：语言模型的能力提升并不只是模型设计本身的胜利，也依赖于我们能否持续获得更多算力、提高算力利用率、并把算法映射到合适的硬件上。于是，理解 GPU 就不再是系统工程师的专长，而是训练大模型的人必须掌握的底层素养。

1.1 从算力扩展谈起

课程先回顾了一个很经典的叙事：为什么我们今天讨论的是 GPU，而不是继续指望 CPU 单线程自然提速。早期芯片性能增长依赖的是 Dennard scaling 与 Moore’s law 的共同作用。晶体管变小，频率可以更高，功耗还能下降，于是单线程性能持续提升。但这条路在 2000 年前后逐渐遇到极限，之后晶体管还在增加，单线程性能却不再等比例增长，计算行业开始转向并行化。

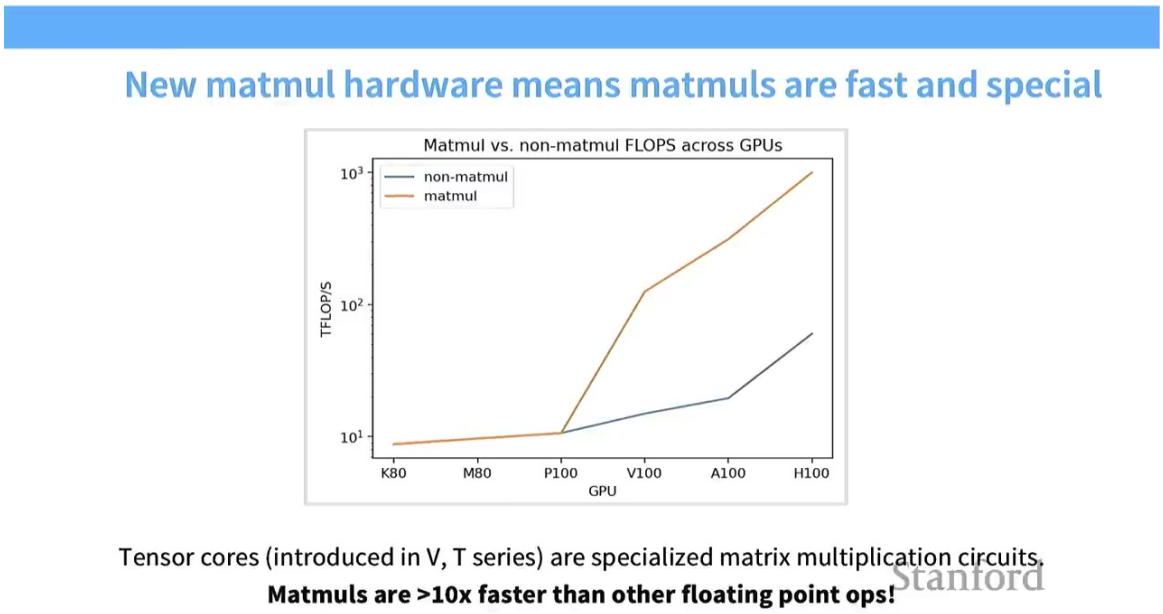


图 2: 从 Dennard scaling 到矩阵乘专用硬件：硬件路线为何转向并行与专用化

来源时间：00:04:41–00:05:26 与 00:21:40–00:22:15。

背景知识

今天深度学习系统里常说的“算力扩展”，其实包含三层含义：更多晶体管、更多并行单元、更高的专用化程度。GPU 和 Tensor Core 的出现，本质上是后两条路线的胜利。

1.2 本章小结

这节课的入口不是硬件百科，而是性能问题。讲者要解释的，是大模型系统里最关键的一件事：硬件结构如何决定算法上限，而优秀算法又如何反过来贴合硬件。

2 GPU 与 CPU：设计目标完全不同

课程接下来给出一个非常重要的对照：CPU 与 GPU 的不同，不只是“谁核多谁核少”，而是设计目标不同。CPU 优先追求低延迟，也就是尽快完成单个任务；GPU 优先追求高吞吐，也就是让大量任务整体更快地完成。

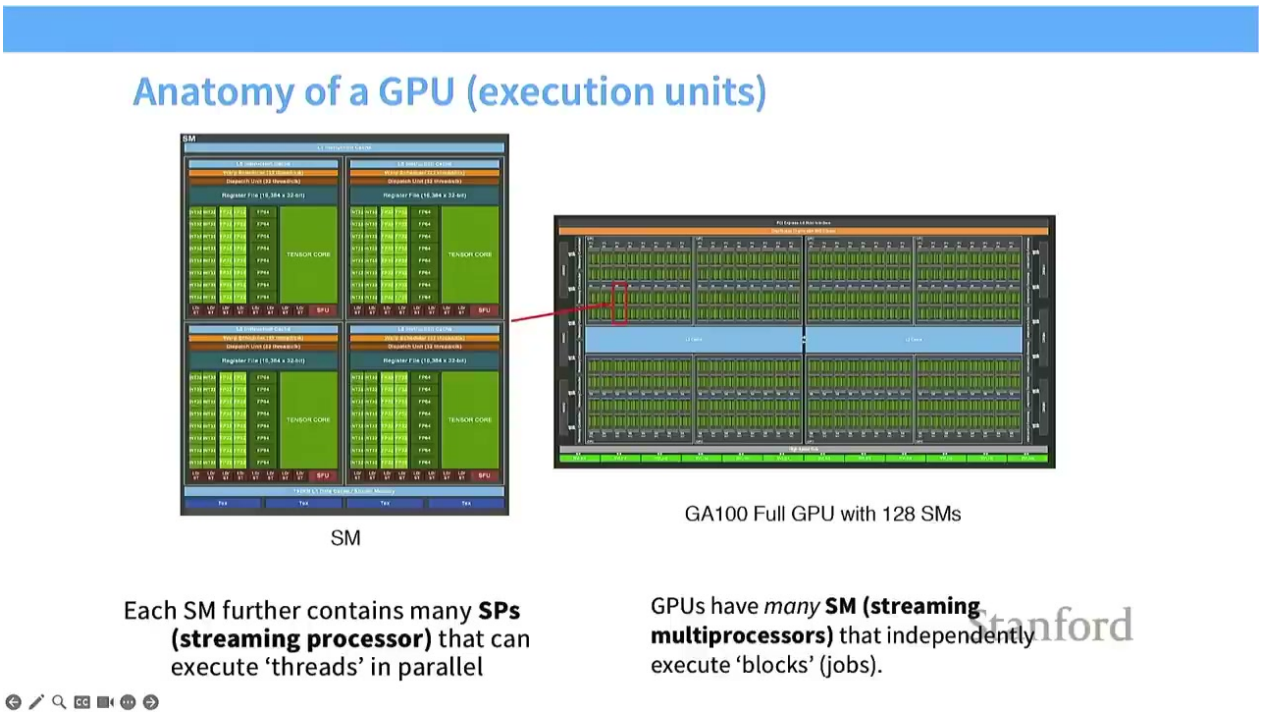


图 3: GPU 的基本解剖：由大量 SM 构成的并行执行阵列

来源时间：00:08:46–00:09:31。

讲者的解释非常形象。对于 CPU 而言，如果有任务 T_1, T_2, T_3, T_4，理想状态是尽快把其中某一个任务完成；而对 GPU 而言，单个任务慢一点无所谓，只要大量任务能并发排队、快速切换、整体吞吐更高就行。于是 GPU 的结构自然会偏向更多更轻量的并行执行单元，而不是把控制逻辑、缓存预测、分支优化做得像 CPU 那么重。

核心区分

CPU 的关键词是 “latency”，GPU 的关键词是 “throughput”。凡是理解不了 GPU 性能现象的地方，回到这两个词，通常就能重新建立直觉。

进一步说，GPU 的原子控制单元是 SM（Streaming Multiprocessor）。程序在 Triton 或 CUDA 中写出来以后，真正的执行粒度会落到这些 SM 上。每个 SM 内部包含控制逻辑和大量执行单元，能够把同一条指令施加到多份不同数据上，从而形成大规模数据并行。

2.1 为什么这对大模型如此重要

大模型的主算子，例如矩阵乘、注意力、卷积，本身就非常适合 “同样的指令、不同的数据” 这种模式。GPU 恰好为这种模式而生。也因此，理解 GPU 不是为了欣赏硬件结构，而是为了理解为什么矩阵乘会成为王者算子，为什么注意力实现会围绕数据搬运和并行粒度展开优化。

常见误解

不要把 GPU 理解成 “很多小 CPU 拼起来”。GPU 的控制逻辑、缓存策略和执行模式都不是 CPU 的简单复制品，它的价值恰恰在于为了吞吐量而刻意放弃了一部分通用性与低延迟能力。

2.2 本章小结

CPU 与 GPU 的差异首先是目标差异，而不是参数差异。理解吞吐优先，后续的 SM、warp、shared memory、coalescing 才会自然连起来。

3 GPU 的内存层次与执行模型

3.1 从物理距离理解内存速度

讲者特别强调一个非常有用的原则：**离 SM 越近的内存越快**。这句话几乎可以当作 GPU 性能直觉的总纲。寄存器、L1、shared memory 在 SM 内部或极近的位置，因此速度最快；L2 仍在芯片上，但更远一些；HBM 则在芯片外部，容量更大但访问更慢。

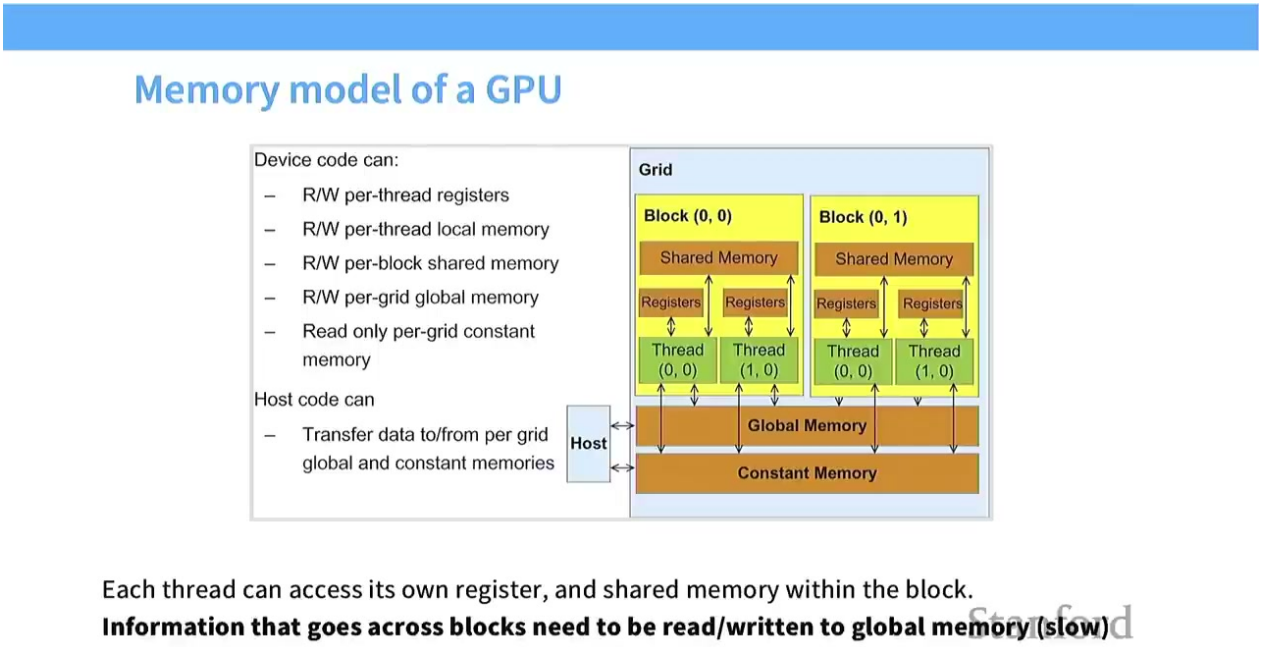


图 4: GPU 内存模型：寄存器、shared memory、global memory 的层次关系

来源时间：00:11:23–00:12:12。

如果只记一句话，那就是：GPU 优化的本质通常不是“多做算术”，而是“少搬数据”。这也是后面所有技巧都会回到的问题。

内存层次的工作原则

寄存器适合线程私有、频繁读写的标量；shared memory 适合同一 block 内反复复用的数据；global memory/HBM 容量大但贵，任何能避免的往返都值得避免。

3.2 Blocks、Warps、Threads

逻辑执行模型上，讲者给出的顺序是 block、warp、thread。

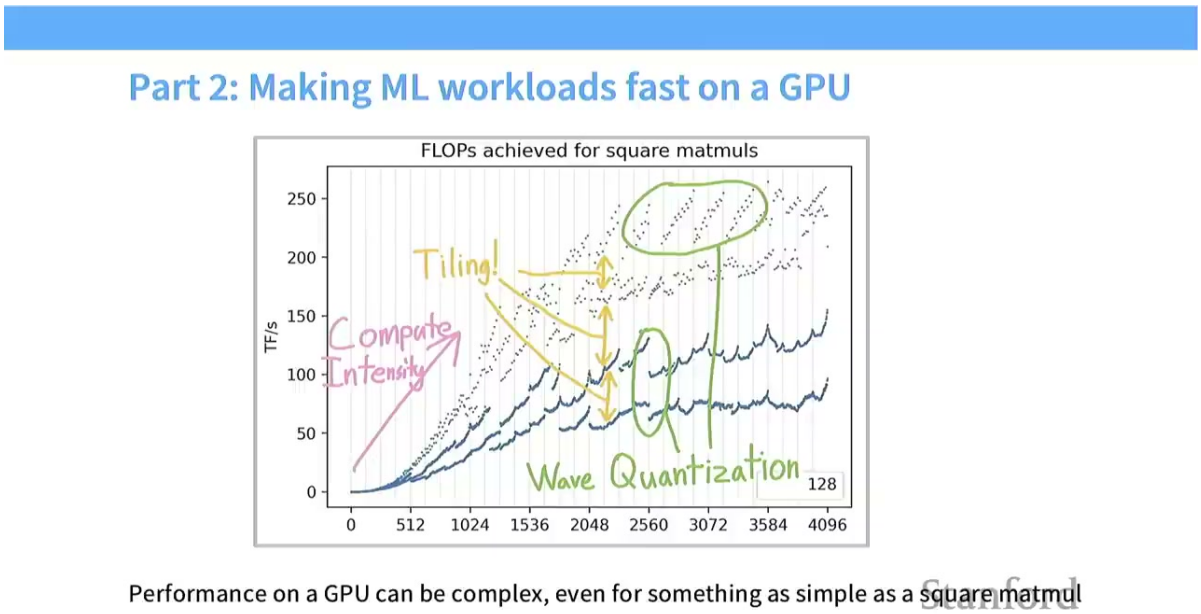


图 5: GPU 的逻辑执行模型：block 分配到 SM，warp 由 32 个线程组成

来源时间：00:13:38–00:15:15。

一个 block 会被分配给某个 SM；block 内又会被切成若干 warp；每个 warp 由 32 个连续编号的线程组成，并以锁步方式执行同一条指令、处理不同数据。于是，程序员虽然写的是“很多线程”，硬件真正调度时看到的关键对象其实是 warp。

为什么 warp 如此重要

后面所有性能问题几乎都与 warp 有关：分支发散看的是 warp 内线程是否走同一路径；memory coalescing 看的是 warp 内线程是否访问相邻地址；吞吐利用率看的是 warp 能否把执行单元喂满。

3.3 一个实用公式：Roofline 直觉

课程中段引入了 roofline model，它是理解“算得慢还是搬得慢”的最好工具。其核心关系可以写成：

$$\text{Performance} = \min\left(P_{\max}, I \cdot B\right)$$

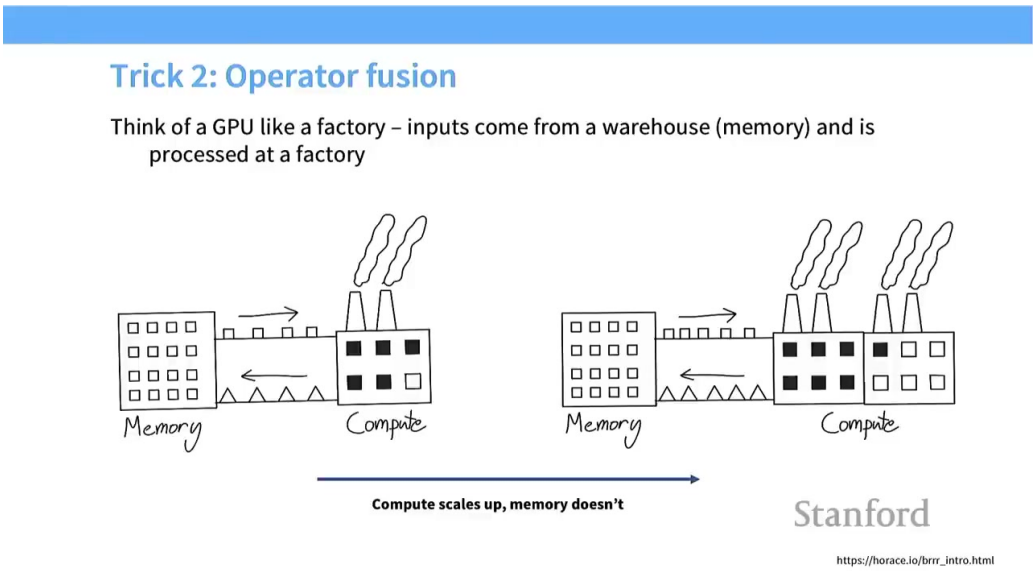


图 6: 课程中的 roofline 视角：吞吐量受算力上限与带宽上限共同约束

来源时间：00:26:31–00:26:44。

符号解释如下：

1. $P_{\max}$：硬件的峰值计算吞吐，例如矩阵乘单元理论上能达到的最高 FLOPS。2. I：算术强度（arithmetic intensity），即每搬运一个字节数据能做多少计算。3. B：可用内存带宽。

这个公式的含义是：如果 I 很低，那么系统受限于内存搬运；如果 I 很高，那么系统才有机会接近峰值算力。于是，想让 GPU 真正快起来，就必须尽量提高算术强度，而不是只盯着 FLOPS 数字。

容易混淆的地方

很多人一看到 GPU 峰值算力很高，就以为程序必然会快。错在忽略了 $I \cdot B$ 这一项。只要内存访问组织得差，实际性能完全可能远低于理论峰值。

3.4 本章小结

GPU 的逻辑执行单位是 block/warp/thread，物理瓶颈则经常由内存层次决定。把这两件事合起来，后面所有优化技巧才真正有落点。

4 让 GPU 真正跑快的四类技巧

课程下半场的重点不是再讲更多硬件部件，而是总结四大优化方向：减少内存访问、让访问方式更友好、提高算术强度，以及把数据搬进更近的内存层次。FlashAttention 的设计后来几乎全部可以映射回这些原则。

4.1 技巧一：算子融合与减少中间写回

讲者用工厂隐喻解释 operator fusion：如果每个算子做完都把中间结果送回“仓库”（全局内存），下一个算子再从仓库取回来，就会造成大量不必要的数据搬运。更好的策略是在一次 kernel 中尽量把相邻操作串起来，在寄存器或 shared memory 里直接继续计算。

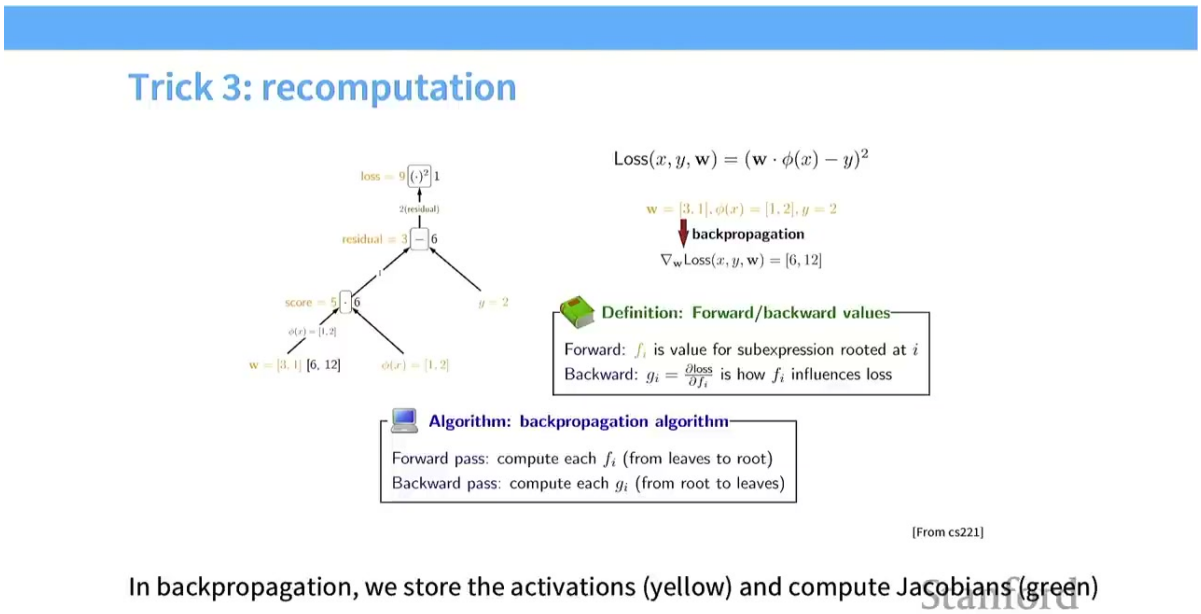


图 7: 算子融合：减少 memory → compute → memory 的往返

来源时间：00:33:40–00:34:20。

4.2 技巧二：重计算（Recomputation）

重计算的思想非常反直觉，但也非常关键。传统做法会在 forward 过程中把中间激活全部存下来，以便 backward 使用；而 GPU 上这些激活的读写本身就可能非常昂贵。于是，在某些场景里，宁可反向时多算几次，也不愿把大量中间结果写入再读回全局内存。

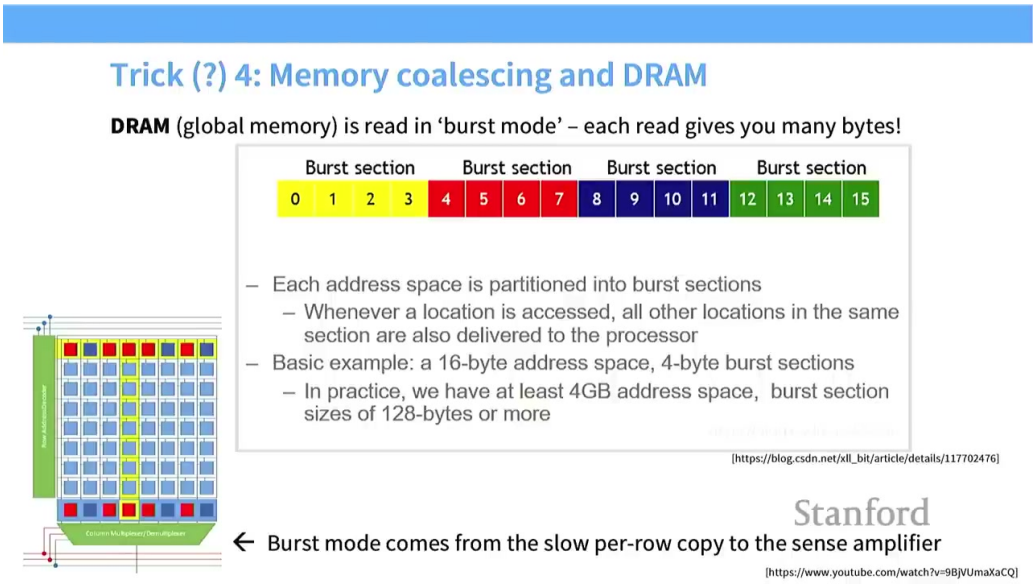


图 8: 重计算示意：少存激活，多做一些重复算术，反而可能更快

来源时间：00:37:29–00:39:55。

讲者给出的例子说明：如果一个计算图没有很高的算术强度，却伴随着大量中间状态读写，那么“减少内存 I/O”往往比“减少 FLOPs”更重要。这正是大模型训练里 activation checkpointing 和 FlashAttention backward 的共同直觉。

重计算的判断标准

当一个中间量很大、写回代价高、且重新计算并不太贵时，重计算通常是值得的。GPU 优化里经常出现“用算术换带宽”的局面。

4.3 技巧三：Memory Coalescing

coalescing 关注的是：warp 内的 32 个线程在同一时刻访问的地址，能否落在相邻的 burst section 中。如果能，硬件就能把很多细碎请求合并成更少、更顺滑的内存事务；如果不能，同样数量的数据会被拆成很多低效读取。

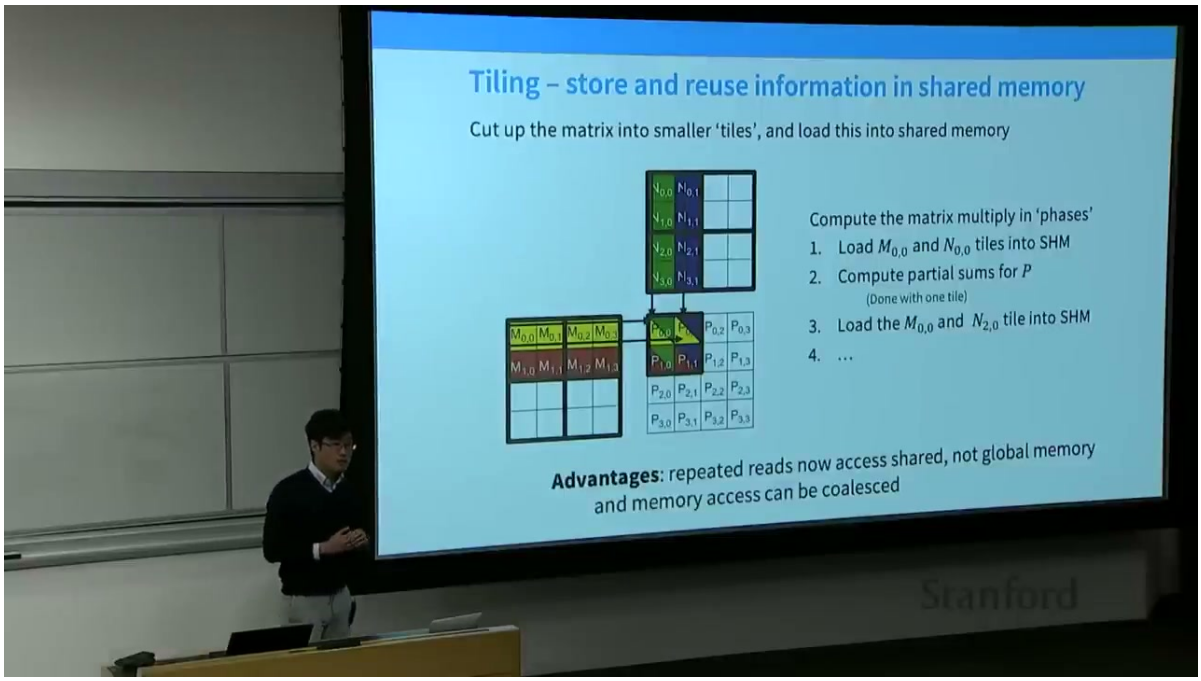


图 9: Memory coalescing：warp 内相邻线程访问相邻地址时，带宽利用率最高

来源时间：00:43:59–00:46:55。

讲者随后把这个原则应用到矩阵乘读取模式上：如果线程遍历方向与内存布局错位，线程会在同一时刻访问很多相距很远的地址，导致访问无法合并；若把线程组织方式与数据布局对齐，则 warp 的访问就会自然 coalesce。

一个非常常见的坑

代码数学上是对的，不代表访问模式就是高效的。对于 GPU 而言，同一个矩阵乘实现，线程索引映射稍有不同，吞吐量可能就差一个量级。

4.4 技巧四：Tiling

tiling 是本课最重要的优化观念之一。它的基本想法是：把大矩阵切成小块，让每次只把当前需要的一小部分数据搬进更近的存储层级（尤其是 shared memory 或 SRAM），在块内完成尽可能多的重复计算，然后再换下一块。

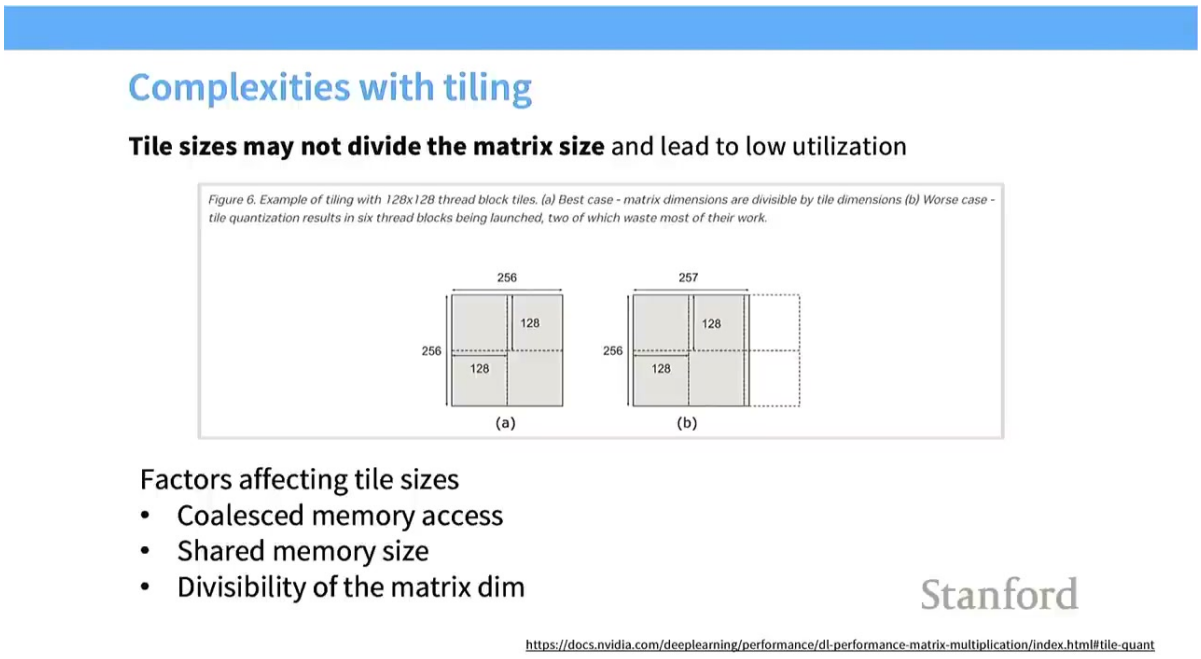


图 10: Tiling：把矩阵切块并搬入 shared memory，以提高复用与算术强度

来源时间：00:47:00–00:49:20。

如果不做 tiling，在矩阵乘中，很多输入会被从 global memory 反复读取；如果做了 tiling，同一块数据被搬进 shared memory 后，可以在块内被多个线程重复使用。于是：

$$I = \frac{\text{FLOPs}}{\text{Bytes moved}}$$

符号解释如下：

1. FLOPs：执行的浮点运算数。2. Bytes moved：在较慢内存层次之间搬运的数据量。

tiling 的作用，就是在不改变数学结果的前提下，显著降低分母的增长速度，从而提高算术强度。

为什么 tiling 几乎无处不在

矩阵乘、卷积、注意力、归约甚至一些图算法，最后都会回到“切块、局部复用、减少外层内存访问”这条主线。它不是某个算子的技巧，而是一种硬件友好的组织原则。

4.5 波数量化（Wave Quantization）

课程还提到一个很有意思但常被忽略的现象：tile 大小如果与 SM 数量不匹配，最后一波工作可能只占用少量 SM，导致利用率突然下降。这种现象被称为 wave quantization。它提醒我们：tile 设计不只是局部缓存问题，也影响全局调度利用率。

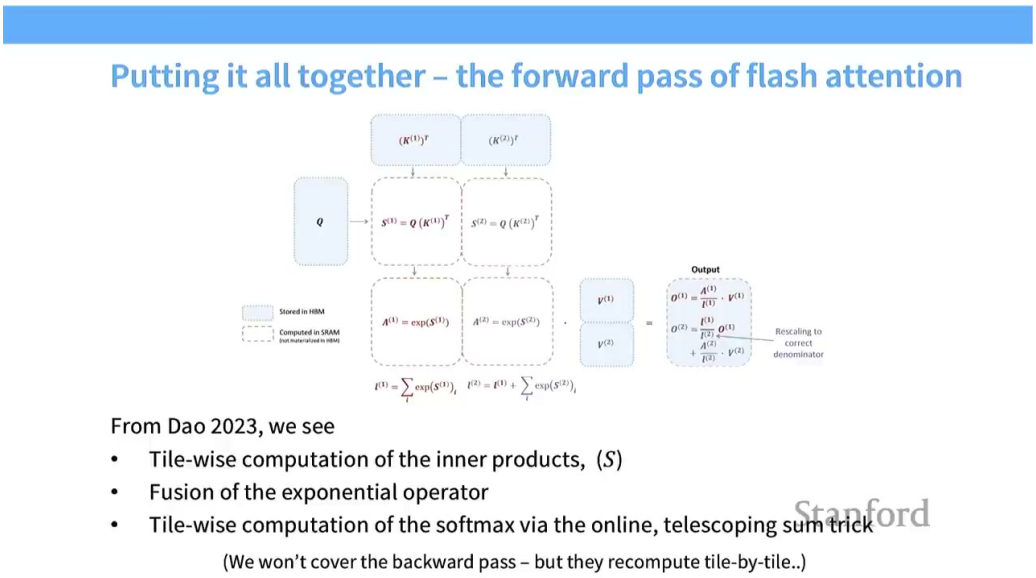


图 11: FlashAttention 的切块视角：tile 不只决定缓存复用，也决定调度波次与利用率

来源时间：01:02:59–01:03:16 与 01:06:45–01:07:37。

4.6 本章小结

讲者归纳出的快算法路径非常统一：少写回、少搬运、让访问可合并、让块内复用更充分。与其说这些是四个零散技巧，不如说它们都是“把昂贵的全局内存访问压缩到最低”的不同实现方式。

5 FlashAttention：把所有原则装进一个算法

这节课最后的高潮，就是把前面看似零散的概念全部收束到 FlashAttention 上。讲者强调，FlashAttention 并不神秘，它的核心仍然是 tiled matrix multiplication，只是把 softmax 这个看上去“全局”的操作，重新组织成了硬件友好的在线计算流程。

5.1 FlashAttention 的硬件视角

讲者先从 FlashAttention 论文中的 Figure 1 切入：Q、K、V 被切成若干小块，$QK^\top$ 的局部块被搬到更近的存储层里做乘法和归约，再与 V 块组合。与朴素注意力相比，关键变化不在数学定义，而在数据流的组织方式。

FlashAttention 的第一性原理

FlashAttention 的本质不是“发明了新的注意力公式”，而是重新安排了注意力的数据搬运路径，让大部分工作在更近、更快的内存层次完成。

5.2 为什么 softmax 是难点

注意力里的 softmax 按行进行，而每一行的归一化因子需要看完整行，这看起来像一个天生依赖全局内存的大操作。FlashAttention 的关键突破，在于把 softmax 写成在线更新形式，使得块与块之间可以增量维护最大值与归一化和，而不必把整个 $n \times n$ 注意力矩阵显式写回 HBM。

可以把它理解为下面的过程：

$$O = \mathrm{softmax}(QK^\top)V$$

符号解释如下：

1. Q：Query 矩阵。2. K：Key 矩阵。3. V：Value 矩阵。4. $QK^\top$：每个 query 与所有 key 的打分。5. $\mathrm{softmax}(\cdot)$：按行归一化，形成注意力权重。6. O：最终输出表示。

朴素实现的问题是，$QK^\top$ 的中间矩阵是 n^2 级别的；FlashAttention 的目标则是避免把这个巨大中间量完整写回全局内存。

5.3 FlashAttention 如何对应前文四大技巧

前面讲的每一条原则，都能在 FlashAttention 中找到对应位置。

1. **Tiling**：把 Q, K, V 分块，块内完成乘法与归约。2. **Coalescing**：让线程按适合矩阵布局的方式搬运连续块。3. **减少写回**：不显式存完整注意力矩阵。4. **Recomputation**：在 backward 中按 tile 重新计算需要的中间量，而不是缓存所有激活。

这也是为什么 FlashAttention 看起来像系统算法

从数学上看，FlashAttention 和标准注意力做的是同一件事；从系统上看，它重新设计了数据路径、内存占用和 kernel 组织方式，因此性能差异会非常大。

5.4 一个简化的 tiled attention 伪代码

```
for q_tile in Q_tiles:
    load q_tile into SRAM
    running_max = -inf
    running_sum = 0
    running_out = 0
    for k_tile, v_tile in KV_tiles:
        load k_tile, v_tile into SRAM
        scores = q_tile @ k_tile.T
        update running_max and running_sum online
        probs = online_softmax(scores, running_max, running_sum)
        running_out += probs @ v_tile
    write running_out to HBM
```

Listing 1: FlashAttention 的简化硬件友好思路

这段伪代码最重要的不是语法，而是执行路径：尽可能把“读块、算块、归约块”的工作留在近端存储里，把写回次数降到最低。

5.5 本章小结

FlashAttention 不是一个孤立技巧，而是这节课全部思想的综合体。你前面理解得越透，越会发现它其实不是神秘黑盒，而是一次非常漂亮的系统工程收束。

6 总结与延伸

讲者在结尾特别强调，真正驱动大模型进步的，不只是更大的参数量，也包括越来越强的硬件与越来越贴合硬件的算法实现。今天最值得带走的结论，不是某张 GPU 芯片图，而是这样一套思考框架：

1. 先问瓶颈在算力还是在内存搬运。2. 再问线程组织是否符合 warp 的执行方式。3. 再问数据访问是否 coalesced、是否能放入 shared memory。4. 最后问能否通过 tiling、fusion、recomputation 来减少 HBM 往返。

全讲的最高层结论

当前 GPU 发展趋势强烈鼓励我们把注意力放在 memory movement 上。真正限制大模型系统速度的，往往不是“不会算”，而是“搬得太多”。

如果把这节课压缩成一句最实用的话，那就是：**优秀的 GPU 算法，本质上是在用更聪明的数据流，换取更少的外层内存访问**。 FlashAttention 只是这条原则最有代表性的成功案例之一。

6.1 拓展阅读

1. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness 2. Horace He 关于 GPU 与性能优化的系列文章 3. Stanford CS336 课程主页